

2. Algoritmos

- Um algoritmo é uma sequência de instruções sem ambiguidade passo a passo que não necessita de nenhum discernimento ou inventividade para executar.
- Os algoritmos são como receitas culinárias. Inventar uma boa receita pode ser uma tarefa difícil e criativa, mas seguir uma receita deveria ser simples e rotineiro.
- Embora informalmente, o conceito de algoritmo seja frequentemente ilustrado pelo exemplo de uma receita, muitos algoritmos são mais complexos. Eles podem repetir passos (fazer iterações) ou necessitar de decisões (tais como comparações ou lógica) até que a tarefa seja completada.
- A maioria de algoritmos pode ser implementada diretamente por programas de computador; os outros algoritmos podem pelo menos em teoria ser simulados por programas de computador.
- Em muitas linguagens de programação, os algoritmos são executados como funções ou *procedures*.
- O conceito de algoritmo foi formalizado em 1936 através máquinas de Turing de Alan Turing e no cálculo lambda de Alonzo Church, que por sua vez formaram os fundamentos da Ciência da Computação.

Algoritmos e linguagens de programação de computadores

- Geralmente, os algoritmos são informalmente descritos em uma linguagem próxima da linguagem natural, mais facilmente compreendida por um ser humano do que por um computador. Um algoritmo pode, na maior parte dos casos, ser implementado em qualquer linguagem de programação.

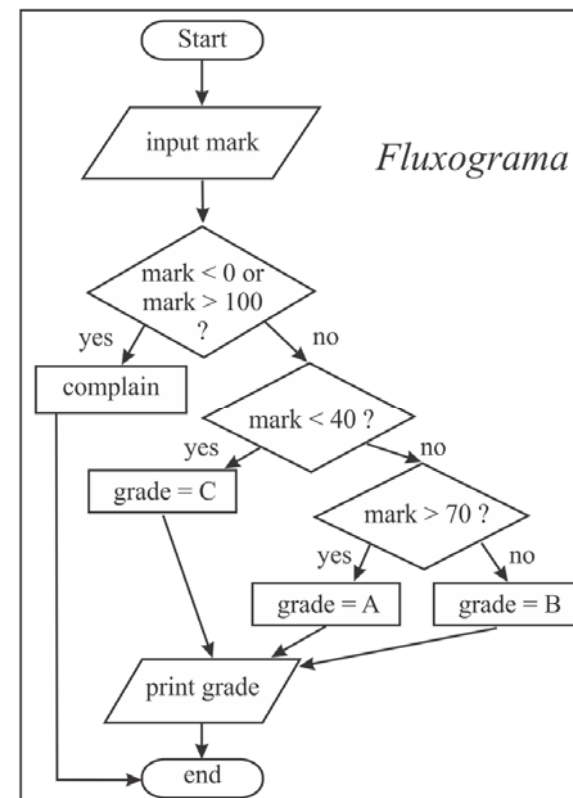
Exercício

Escreva um algoritmo para fazer um sanduíche misto quente, dados os ingredientes.

Desenvolvimento de Algoritmo e Refinamento Passo a passo

Desenvolvimento de Algoritmo

- Para um computador realizar alguma tarefa, tem receber um programa, que é uma implementação de um algoritmo. O programa é expresso em linguagem de programação, no entanto, é possível (e desejável) de falar e raciocinar sobre algoritmos em um nível mais alto.
- As notações mais utilizadas para o desenvolvimento de algoritmos são **fluxogramas** e **pseudocódigo**. Estes são independentes da linguagem de programação a ser utilizada para implementar o algoritmo.



Pseudocódigo

1. input mark
2. complain if mark below 0 or above 100
3. determine grade from mark: ($>70 = A$; $<40 = C$; otherwise B)
4. print grade

Estas são ferramentas para auxiliar no design / refinamento / depuração de algoritmos.

- Um dos problemas encontrados ao escrever programas é a de **precisão**. Um problema comum entre os algoritmos é que o processo descrito é *quase* o desejado, mas não exatamente.
- Outra falha comum é que sua execução normalmente resulta no processo desejado, mas, em determinadas circunstâncias (imprevistas ou ignorados pelo criador) não.
 - Por exemplo, considere o seguinte algoritmo para calcular o tempo de voo de um avião usando informações do voo:

Olhe a hora de partida

Olhe a hora de chegada

Subtraia a hora de partida da hora de chegada

- Outra propriedade necessária de um algoritmo é que cada passo possa efetivamente ser realizado - em outras palavras, o algoritmo tem que ser executável.
- A maioria dos processos deve terminar!
- Assim, o designer de um algoritmo deve assegurar:
 - A sua **precisão** (sem ambiguidades);
 - Que seja capaz de lidar com **todas as circunstâncias possíveis**;
 - Que ele seja **executável**;
 - Ele **termine**.
- Também tem que se preocupar com **Eficiência**.

Refinamento Passo a passo

- Divida um problema complexo em um número de passos mais simples, cada uma das quais pode ser resolvido por um algoritmo que é menor e mais simples do que aquele necessário para resolver o problema completo.
- O **refinamento** do algoritmo continua desta forma até que cada passo seja suficientemente detalhado.
- **Exemplo: fazer chá.** Suponha que temos um robô que realiza tarefas domésticas. Desejamos para programar o robô para fazer uma xícara de chá. Uma primeira tentativa de um algoritmo poderia ser:
 - 1. Colocar as folhas de chá no bule**
 - 2. Ferver água**
 - 3. Colocar a água no bule**

4. Esperar 5 minutos

5. Colocar o chá na xícara

- Estes passos provavelmente não são suficientemente pormenorizados para o robô. Vamos, portanto, refinar cada passo em uma sequência de passos menores:

1. Colocar as folhas de chá no bule

poderia ser refinado para

1.1 Abrir a caixa de chá

1.2 Retirar uma colher de folha de chá

1.3 Colocar o conteúdo da colher no bule

1.4 Fechar a caixa de chá

Semelhantemente:

2. Ferver a água

poderia ser refinado para

2.1 Encher a chaleira com água

2.2 Ligar o fogo do fogão

2.3 Colocar a chaleira no fogo

2.4 Esperar até a água ferver

2.5 Desligar o fogo

Suponha que os cinco passos originais foram refinados em sub-algoritmos, sempre que necessário (por exemplo, não era necessário para refinar passo 4, porque é simples o suficiente para executar diretamente para o robô).

Alguns dos sub-algoritmos necessitam de um refinamento adicional. Por exemplo, o passo

2.1. Encher a chaleira com água

poderia ser refinado para

2.1.1. Colocar a chaleira debaixo da torneira

2.1.2. Abrir a torneira

2.1.3. Esperar a chaleira encher

2.1.4. Fechar a torneira

- Outros passos podem também exigir um refinamento adicional. Após uma série de refinamentos o robô será capaz de executar todas as etapas.
- O programa é, então, construído por traduzir o refinamento final de cada etapa em declarações de programa de computador.

Algoritmo Original

1. Colocar as folhas de chá no bule

2. Ferver água

3. Colocar a água no bule

4. Esperar 5 minutos

5. Colocar o chá na xícara

Primeiro Refinamento

1.1 Abrir a caixa de chá

1.2 Retirar uma colher de folha de chá

1.3 Colocar o conteúdo da colher no bule

1.4 Fechar a caixa de chá

2.1 Encher a chaleira com água

2.2 Acender o fogo do fogão

2.3 Colocar a chaleira no fogo

2.4 Esperar até a água ferver

2.5 Desligar o fogo

3.1 Derramar a água da chaleira até encher o bule

5.1 Derramar o chá da bule até encher a xícara

Segundo Refinamento

1.1.1 Tirar a caixa de chá da prateleira

1.1.2 Retirar a tampa da caixa

1.4.1 Colocar a tampa na caixa

1.4.2 Colocar caixa de chá de volta na prateleira

2.1.1. Colocar a chaleira debaixo da torneira

2.1.2. Abrir a torneira

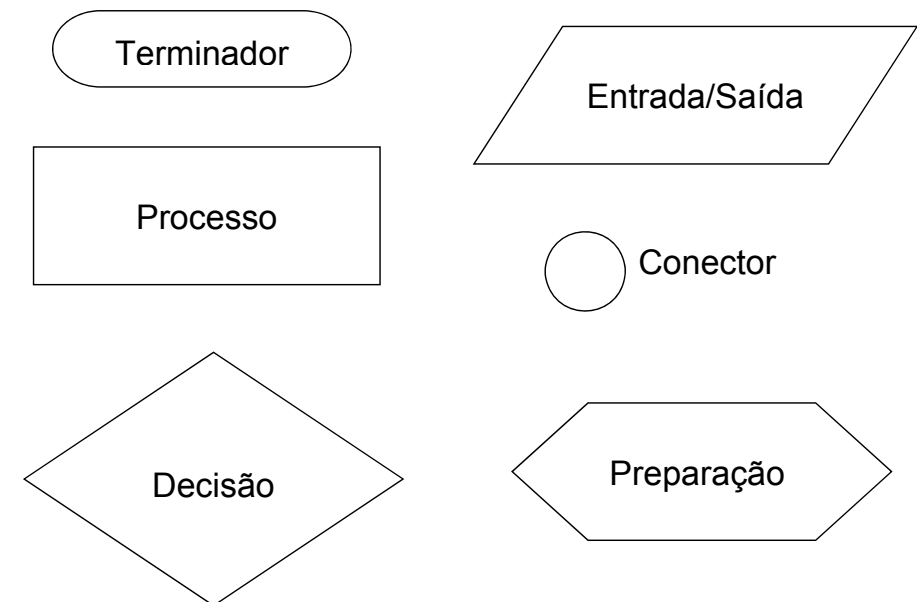
2.1.3. Esperar a chaleira encher

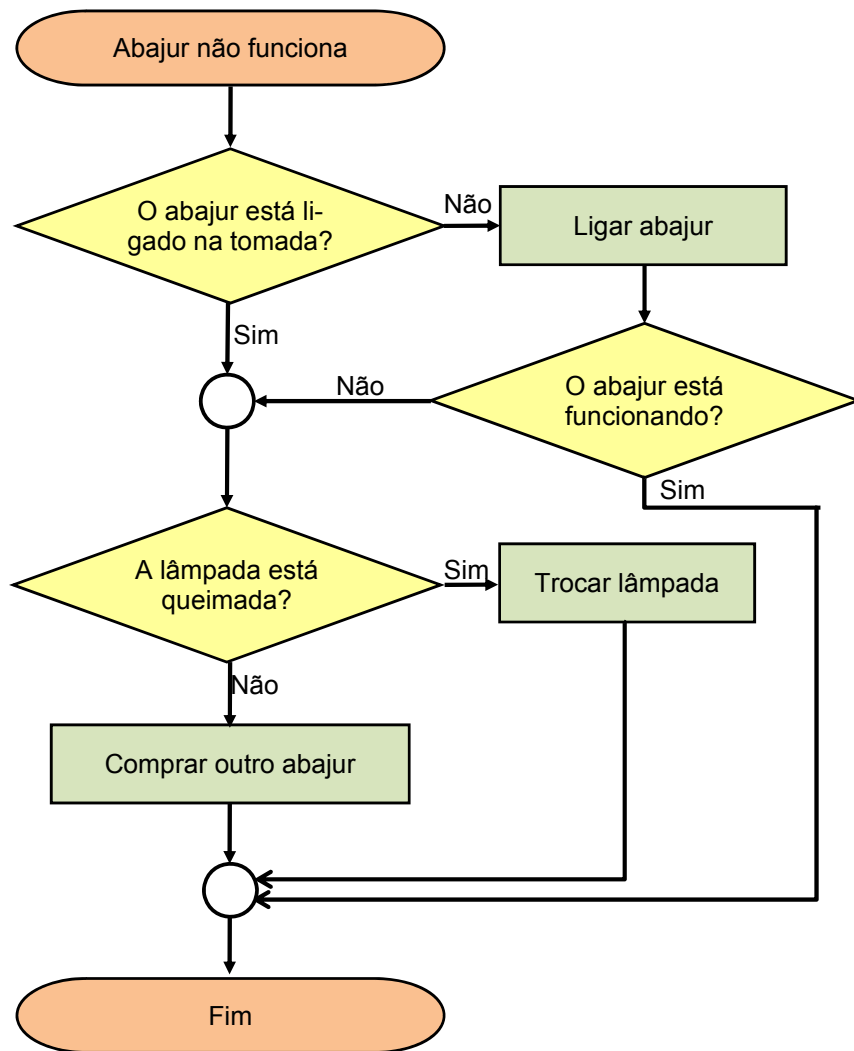
2.1.4. Fechar a torneira

2.4.1 Esperar a chaleira apitar

Fluxograma

Um fluxograma é um tipo de diagrama e pode ser entendido como uma representação esquemática de um processo, muitas vezes feita através de gráficos que ilustram de forma descomplicada a transição de informações entre os elementos que o compõem. Podemos entendê-lo, na prática, como a documentação dos passos necessários para a execução de um processo qualquer. Geralmente contêm o início, fim, entradas, saídas, os possíveis caminhos e as decisões que levam a estes possíveis caminhos.





Pseudocódigo

Pseudocódigo é uma forma genérica de escrever um algoritmo, utilizando uma linguagem simples (nativa a quem o escreve, de forma a ser entendido por qualquer pessoa) sem necessidade de conhecer a sintaxe de nenhuma linguagem de programação. É, como o nome indica, um pseudocódigo e, portanto, não pode ser executado num sistema real (computador) — de outra forma deixaria de ser *pseudo*.

Exemplo

```

SE a tomada estiver desligada
  Ligue a tomada
  SE o abajur estiver funcionando
    TERMINE
  FIM SE
FIM SE
SE a lâmpada estiver queimada
  Troque a lâmpada
SENÃO
  Compre outro abajur
FIM SE
  
```

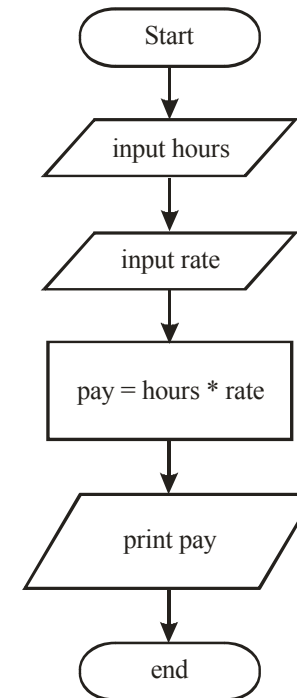
O pseudocódigo é uma espécie de linguagem natural estruturada para descrever algoritmos. Ele permite a quem está escrevendo o algoritmo que se concentre na lógica sem se distrair com detalhes da sintaxe da linguagem de programação. Ao mesmo tempo, o pseudocódigo tem de ser completo. Ele descreve a lógica inteira do algoritmo para que sua implementação se torne uma tarefa mecânica de traduzir linha a linha em código fonte.

Aqui estão algumas orientações gerais para verificar o pseudocódigo:

1. *Imitar o bom código e a linguagem natural.* Utilização dos aspectos de ambos os sistemas significa aderir às regras de estilo de ambos em algum grau.
2. *Ignorar detalhes desnecessários.* Se você se estiver incomodando com a colocação de vírgulas, você está usando detalhe demais.

3. *Não repisar o óbvio.* Em muitos casos, o tipo de uma variável é claro do contexto.
4. *Tirar proveito de formas abreviadas de programar.* Utilização de if-then-else ou estruturas que fazem um laço é mais conciso do que escrever o equivalente em português (inglês).
5. *Não perder de vista o modelo subjacente.* Deve ser possível se perceber através de seu pseudocódigo o modelo que está por trás dele; se não, está escrito em um nível demasiado alto.
6. *Verificar o equilíbrio.* Se o pseudocódigo for difícil para uma pessoa ler ou difícil de traduzir para o código que funcione, então algo está errado com o nível do detalhe que você decidiu usar.

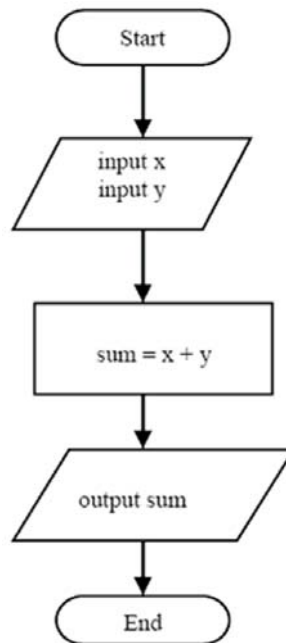
Exemplos



Cálculo do Salário

Começo
Ler horas
Ler taxa
salário = horas * taxa
Imprimir salário
Fim

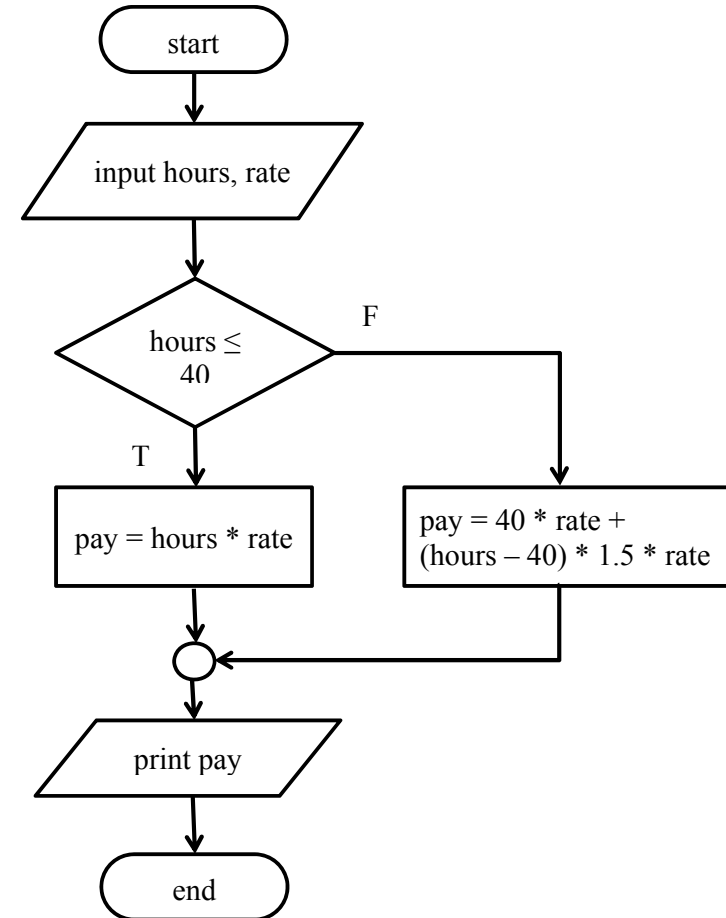
Soma de 2 Números



```

Começo
Ler x, y
soma = x + y
Imprimir soma
Fim
  
```

Cálculo do Salário com Hora Extra – Seleção



```

Começo
Ler horas, taxa
se horas ≤ 40 então
    salário = horas * taxa
caso contrário
    salário = 40 * taxa + (horas - 40) * taxa * 1.5
Imprimir salário
Fim
  
```

Exercício

Escreva um algoritmo para fazer N sanduíches mistos.

As Partes de um Algoritmo

Variáveis: Valores com nome

Parâmetros: Entradas com nome

Condicionais: Lidando com diferentes condições

- se *alguma condição ocorrer* então *faça algo*
- se *alguma condição ocorrer* então *faça algo* caso contrário *faça outra coisa*

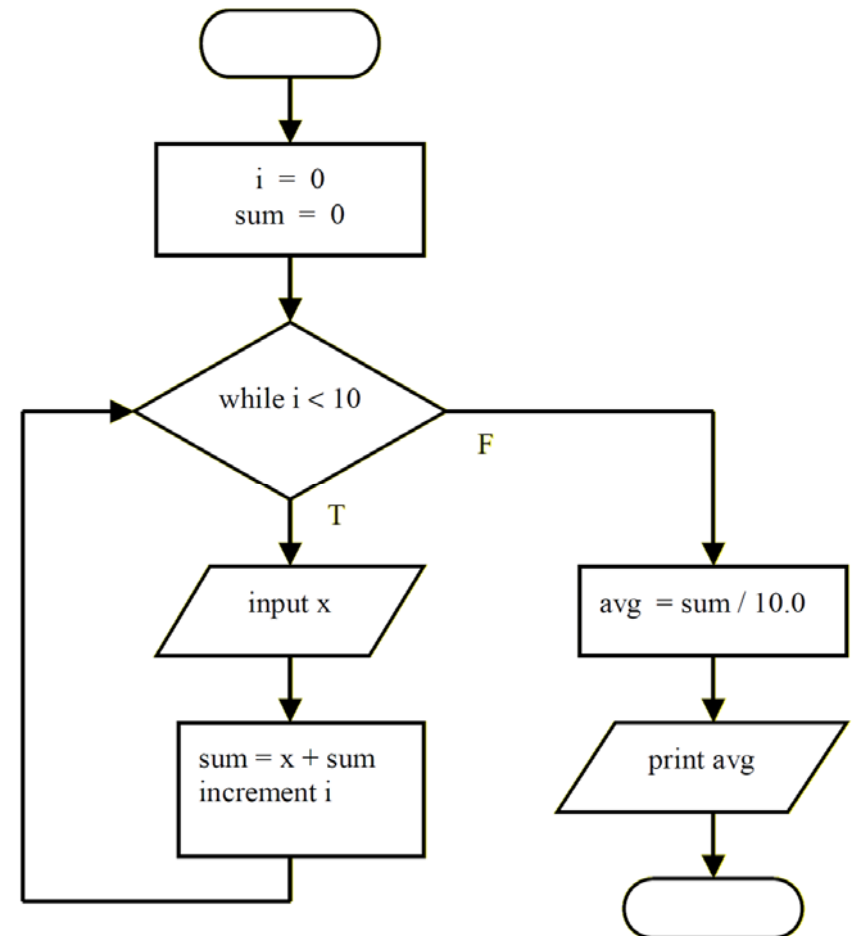
Repetição (iteração)

- repita *alguma ação* até que *alguma condição aconteça*
- repita *alguma ação* enquanto *alguma condição aconteça*
- repita *alguma ação* *N* vezes

Sub-rotinas: Algoritmos Auxiliares

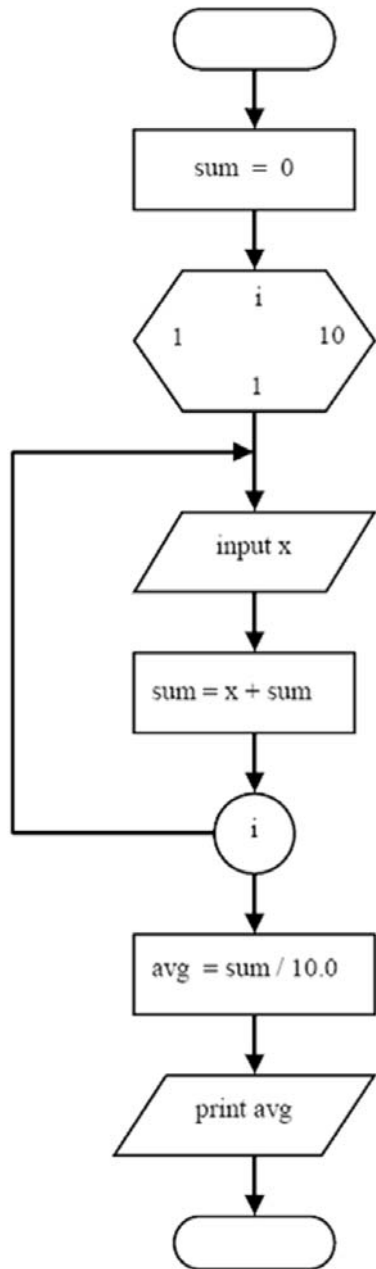
Recorrência (recursão): Ajudando-se

Média de 10 Números – iteração com um laço “while”



```
Começo
i = 0
soma = 0
enquanto i < 10
    ler x
    soma = soma + x
    i = i + 1
média = soma / 10.0
imprimir média
Fim
```

Média de 10 Números iteração com um laço "for"



Começo
 soma = 0
 para i = 1 até 10
 ler x
 soma = soma + x
 média = soma / 10.0
 imprimir média
 Fim

Exemplo de um algoritmo que lê 10 números e calcula a média apenas dos números não negativos.

```

S <- 0;
C <- 0;
PARA i<- 1 ATÉ 10 FAZ
  LER (a);
  SE a >=0 ENTÃO
    S <- S+a
    C <- C+1
  FIM SE;
FIM PARA;
SE C >0 ENTÃO
  MD <- S/C
  ESCREVER ("A média é:", MD)
CASO CONTRÁRIO
  ESCREVER ("Nenhum número válido")
FIM SE;
  
```

Os laços de contagem iteram um número fixo de vezes. Entretanto, em muitos casos não sabemos quantas vezes queremos fazer algo. Muitas vezes depende dos dados fornecidos ao programa. Vamos modificar o nosso problema de ler e computar a média de 10 de números para calcular a média de uma quantidade indefinida de números.

```

ATRIBUA zero A média
ATRIBUA zero A contador
ATRIBUA zero A total
FAÇA
  LEIA número
  SE (fim-de-dados) SAIA
  INCREMENTE contador de 1
  ATRIBUA total + número A total
FIM FAÇA
SE ( contador > 0 ) ENTÃO
  média = total / contador
  MOSTRE média
FIM SE
  
```

Encontrar o maior número em uma lista de números. A solução necessariamente requer que se veja cada número na lista, mas apenas uma vez cada um. Disto segue um algoritmo simples, que pode ser explicitado como:

1. Supor que o primeiro item é o maior.
2. Ver cada um dos itens restantes na lista e se for maior do que o maior item por enquanto, registre-o.
3. O último item registrado é o maior na lista quando o processo terminar.

Aqui está uma codificação mais formal do algoritmo em pseudocódigo:

Algoritmo MaiorNúmero

Entrada: Uma lista não vazia de números L .
Saída: O maior número na lista L .

```
maior ←  $L_0$ 
para cada item na lista  $L_{\geq 1}$ , faça
    se item > maior, então
        maior ← item
retorne maior
```

Fatorial

- 1: programa fatorial iterativo
- 2: Leia n
- 3: Se $n < 0$
- 4: imprima "O valor de n é inválido"
- 5: termine
- 6: $fat := 1$
- 7: para $k := 2$ até n faça
- 8: $fat := k * fat$
- 9: imprima fat
- 10: fim fatorial iterativo

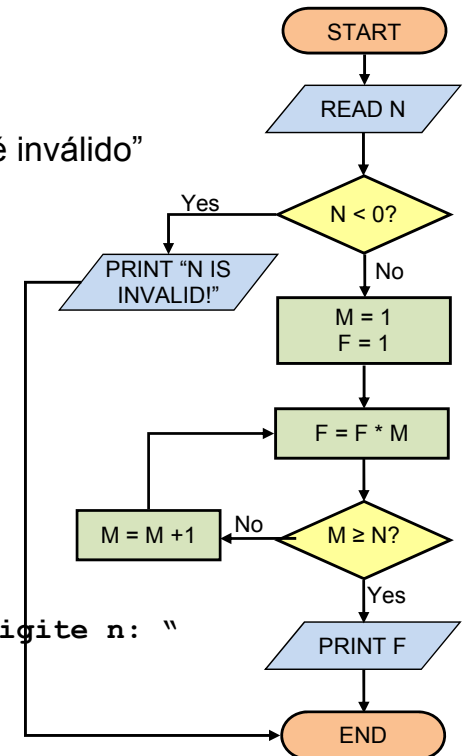
Fortran 90

```
PROGRAM fat
  IMPLICIT none
  INTEGER :: i, n, prod

  PRINT *, " Por favor, digite n: "
  READ *, n
  prod = 1
  DO i=2,n
    prod = i*prod
  ENDDO
  PRINT *, n, "!= ", prod
END PROGRAM fat
```

C

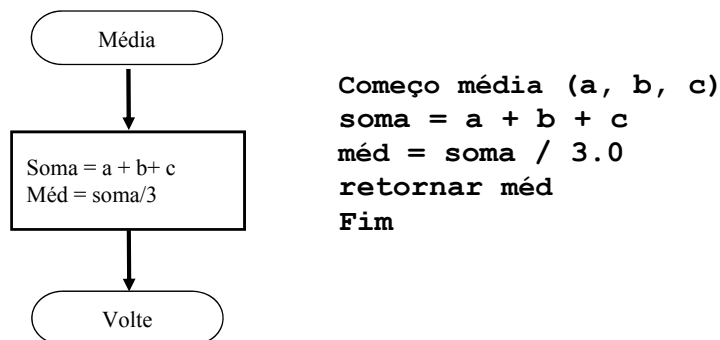
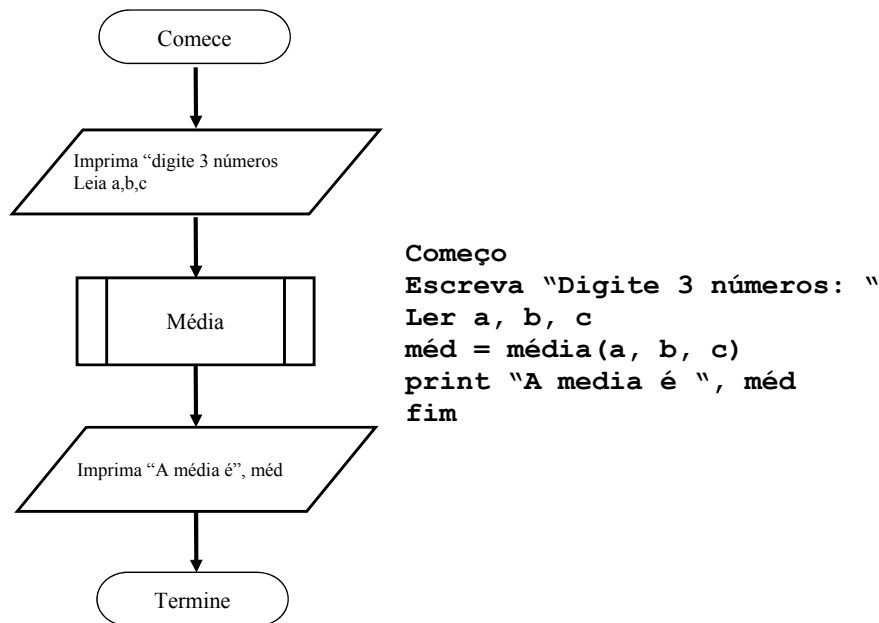
```
#include <stdio.h>
main (void)
{
    int i, n, prod;
    printf ("Por favor, digite n: ");
    scanf ("%d",&n);
    prod = 1;
    for (i=2; i <= n; i++) prod *= i;
    printf ("%d!= %d\n",n,prod);
}
```



Sub-rotinas: Algoritmos de Auxiliares Denominados

Podemos escrever algoritmos adicionais de ações comuns e usá-los como a parte do nosso algoritmo mais amplo. Também podemos usá-los em outros algoritmos. Chamamos esses algoritmos auxiliares “sub-rotinas”.

Fluxograma para uma Function ou Subroutine



Recorrência (recursividade): Ajudando-se

Recursão em programação define uma função em termos dela mesma. Este tipo de algoritmo se aplica quando é possível se dividir um problema em subproblemas do mesmo tipo.

Uma maneira simples de entender processos recursivos:

1. Terminamos? Se sim, retorne os resultados. Sem esta *condição de término* a recursão continuaria indefinidamente.
2. Se não, simplifique o problema, resolva o(s) problema(s) mais simples e monte os resultados para obter a solução do problema original. Então retorne a solução.

Humor Recursivo

“Definição” de recursão

Recursão

Veja “Recursão”

No livro “The C Programming Language” de Kernighan and Ritchie encontra-se a seguinte entrada no índice que se encontra na página 269:

recursão 86, 139, 141, 182, 202, 269

Uma variação da “definição” de recursão

Recursão

Se você não entendeu ainda, Veja “Recursão”

Siglas recursivas

GNU = GNU's Not Unix

LAME = LAME Ain't an MP3 Encoder

PHP = PHP: Hypertext Preprocessor

PINE = PINE Is Nearly Elm, originalmente; PINE agora oficialmente “Pine Internet News and E-mail”

Recursivo

Função Fatorial (inteiro N)

```
Se N < 0 então
  Retorne "Argumento inválido"
Caso contrário
  Se N = 0 então
    Retorne 1
  Caso contrário
    Retorne Fatorial(N-1) * N
Fim se
Fim se
Fim função
```

Iterativo

FUNÇÃO Fatorial (inteiro N)

```
INTEIRO resultado_temp
SE N < 0 ENTÃO
  retorne "Argumento inválido"
FIM SE
resultado_temp = 1
PARA i = 1 ATÉ N FAÇA
  resultado_temp *= i
FIM PARA
RETORNE resultado_temp
FIM FUNÇÃO
```

Recursivo

Para fazer N *sanduíches mistos*

Se N é 0 então *condicional*

Comemore, você terminou!

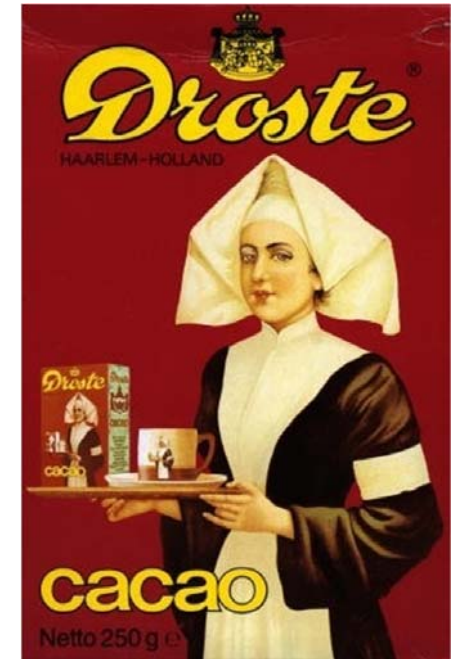
Caso contrário

Faça um *sanduíche* *sub-rotina*

Faça N-1 *sanduíches mistos* *recursão*

Recursividade versus Iteração

Enquanto a recursividade pode simplificar a solução de um problema, muitas vezes resultando em um código fonte mais curto, mais facilmente compreensível, é muitas vezes menos eficiente, tanto em tempo quanto em espaço, do que soluções iterativas.



Classificação por implementação

Um modo de classificar algoritmos é por meios de implementação.

- **Recursividade** ou iteração
- Serial ou paralelo
- Determinista ou não determinista.
- Exato ou aproximado

Classificação por metodologia ou paradigma de projeto

Outro modo de classificar algoritmos é pela sua metodologia de desenho ou paradigma.

- Dividir e conquistar
- Programação dinâmica
- Método greedy (ganancioso)
- Programação linear
- Buscar e enumerar
- O paradigma probabilístico e heurístico. Os algoritmos que pertencem a esta classe se encaixam na definição de um algoritmo mais livremente.
 1. Algoritmos probabilísticos
 2. Algoritmos genéticos
 3. Algoritmos heurísticos